

1. Types of ML:

① Supervised Learning: Input: Data with desired output

Output: A function (i.e. model) that maps data to the desired output.

② Unsupervised Learning: Input: Data (No desired output)

Output: Undelying patterns in data (Clustering)

③ Reinforcement Learning:

Input: Sequence of states, actions, and delayed rewards.

Output: Action strategy (A rule that maps the environment to action).

2. Three components in ML: Task, Performance Measure, Experience

3. Three ways of handling missing values: Dropping and imputation (1. Take average value 2. Take a value outside the normal range)

4. Data formatting:

① Convert non-numerical data into numerical form

② Normalization: { Min-max normalization: $X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$
Z score standardization: $X_{scaled} = \frac{X - \overset{\text{mean}}{M}}{\underset{\text{standard deviation}}{\sigma}}$

5. Feature selection: Pearson's correlation coefficient (r)

(Measures linear relationships between 2 numerical variables)

Way 1: (Regression)

- Compute r between each input feature and the target

output in the training set

- Select features with high absolute r .

Way 2: (Regression, Classification)

- Compute r between every pair of input features in the training set
- Remove features with high absolute r .

b. Data partitioning: Training, Validating, Testing

What is Machine Learning?

A computer program is said to learn if its performance at task **T**, as measured by **P**, **improves** with **experience** **E**.

- Tom Mitchell

e.g., Google Maps finding the fastest route based on current traffic?

Google Maps predicting traffic jams based on historical traffic patterns?

millions of past trips from
other users

Experience
(**E**)

traffic jam
prediction

Task
(**T**)

Prediction accuracy

Performance
Measure
(**P**)

1. Linear Regression:

Least squares regression

① Learning : Minimize : $J(w) = \sum_{i=1}^N (f_w(x_i) - y_i)^2 = \sum_{k=1}^n (X w_k - y_k)^T (X w_k - y_k)$

The optimal value w^* ($y = ax + b$) should have zero gradient:
 $\frac{dJ(w)}{dw} = 0$ If $X^T X$ is invertible, $w^* = (X^T X)^{-1} X^T y$

② Prediction: $f_{w^*}(X_t) = X_t w^*$

2. Polynomial Regression

① Total No. parameters for n -th order polynomial model with d input variables is $\binom{n+d}{d} = \frac{(n+d)!}{n! d!}$

$$\textcircled{2} f_w(x) = f_w(P(x)) = P^T w$$

$$\text{Minimize: } J(w) = (Pw - y)^T (Pw - y)$$

$$P = \begin{bmatrix} 1 & X_{1,1} & - & - & X_{1,d} & - & - & - & X_{1,i} & X_{1,j} & - & - & X_{1,i} & X_{1,j} & X_{1,k} & - \\ \vdots & \vdots & & \vdots & \vdots & & \vdots & & \vdots & & & & \vdots & & \vdots & \\ 1 & X_{N,1} & - & - & X_{N,d} & - & - & - & X_{N,i} & X_{N,j} & - & - & X_{N,i} & X_{N,j} & X_{N,k} & - \end{bmatrix}$$

$$\text{Solution: } w^* = (P^T P)^{-1} P^T y, \text{ if } P^T P \text{ is invertible}$$

$$\text{Prediction: } F_{w^*}(P(X_t)) = P_t w^*$$

3. Ridge Regression (If $X^T X$ or $P^T P$ is not invertible)

$$\textcircled{1} \text{Minimize: } J(w) = (Pw - y)^T (Pw - y) + \lambda w^T w \quad \lambda \text{ 为正则项}$$

$$\textcircled{2} \text{Since } (P^T P + \lambda I) \text{ is always invertible, } w^* = (P^T P + \lambda I)^{-1} P^T y$$

$\textcircled{3}$ Can convert constraint into unconstraint:

Primary form: $\underset{w}{\operatorname{argmin}} (Pw - y)^T (Pw - y)$, subject to $w^T w \leq C$

Dual form: $\underset{w}{\operatorname{argmin}} (Pw - y)^T (Pw - y) + \lambda w^T w$

④ Prediction: $Fw^*(P(X_t)) = P_t w^*$

⑤ Regularization: Use λ (penalty) to generalize model.

$\lambda \uparrow \rightarrow$ strong penalty $\rightarrow$ simpler model

$\lambda \downarrow \rightarrow$ weaker penalty $\rightarrow$ more flexible model

Reason: Overfit!!! Order 太高, 不利于得出结果

⑥ λ (hyperparameter) 减少 order.

Ridge Regression

➤ Ridge Regression = Regression + L2 regularization

Shrinks the regression coefficients w by imposing a penalty to their size.

Find the optimal value $\mathbf{w}^*$ which minimizes:

$$J(\mathbf{w}) = \sum_{i=1}^N (f_{\mathbf{w}}(\mathbf{p}(\mathbf{x}_i)) - y_i)^2 + \lambda \sum_i w_i^2$$

L2 regularization:
 $\sum_i w_i^2 = \|\mathbf{w}\|_2^2$,
squared L2 norm of $\mathbf{w}$

$$= (\mathbf{P}\mathbf{w} - \mathbf{y})^T (\mathbf{P}\mathbf{w} - \mathbf{y}) + \lambda \mathbf{w}^T \mathbf{w}$$

Handwritten notes in red:
L2 为正则化
λ ↑
w ↓

(Hyperparameter)

$\lambda > 0$ is a complexity/regularization parameter that controls the amount of shrinkage: the larger the value of λ , the greater the amount of shrinkage.

Evaluation Metrics for Regression

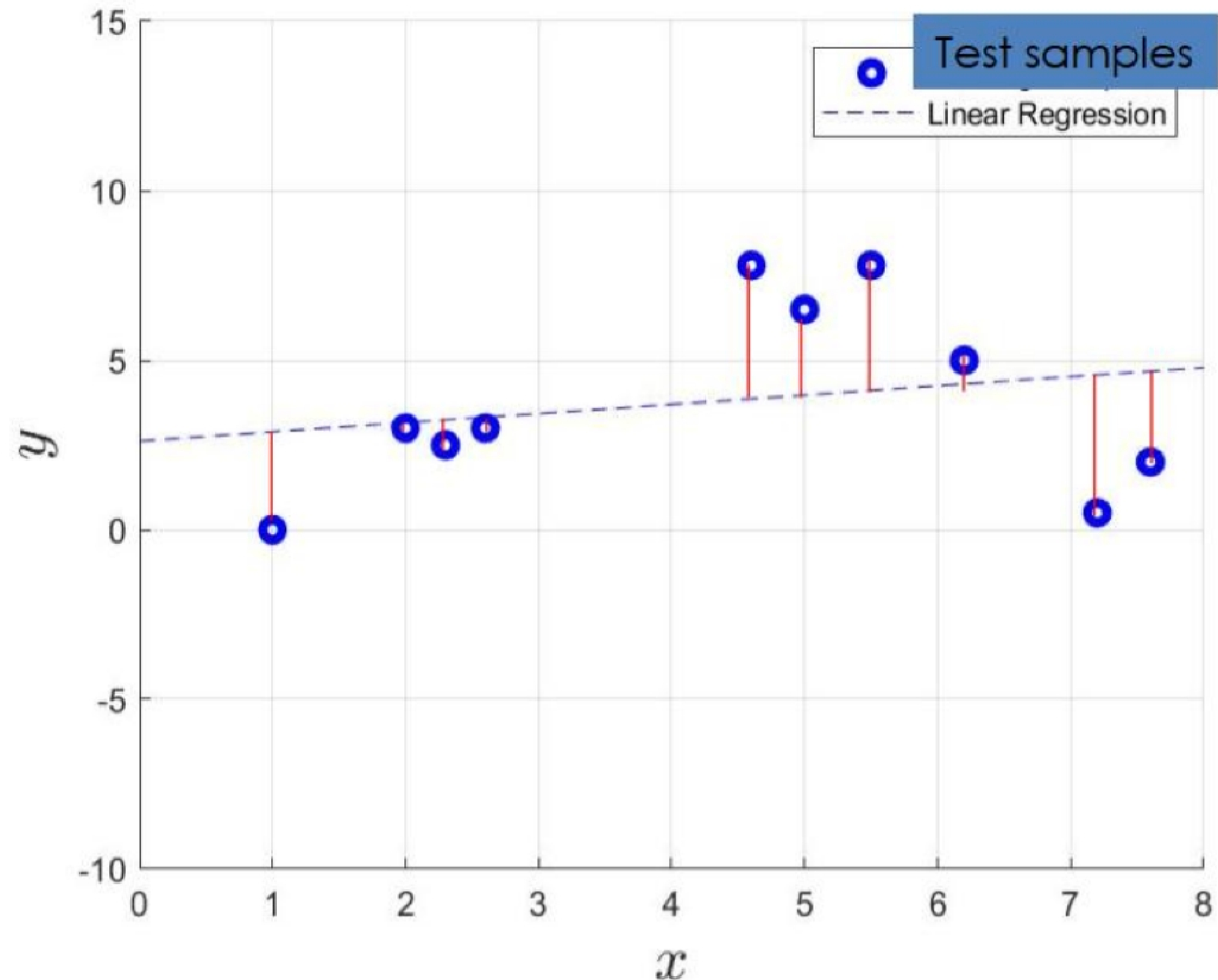
Mean Square Error

$$(\text{MSE} = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n})$$

Mean Absolute Error

$$(\text{MAE} = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{n})$$

where y_i denotes the target output and $\hat{y}_i$ denotes the predicted output for sample i .



```
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
mae = mean_absolute_error(y_true, y_pred)
```

```
mse = mean_squared_error(y_true, y_pred)
```

Summary

✓ Linear Regression

Scalar output for each sample

-Learning: $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

-Prediction: $\mathbf{f}_{\mathbf{w}^*}(\mathbf{X}_t) = \mathbf{X}_t \mathbf{w}^*$

Vector output for each sample

-Learning: $\mathbf{W}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$

-Prediction: $\mathbf{F}_{\mathbf{W}^*}(\mathbf{X}_t) = \mathbf{X}_t \mathbf{W}^*$

✓ Polynomial Regression

Scalar output for each sample

-Learning: $\mathbf{w}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{y}$

-Prediction: $\mathbf{f}_{\mathbf{w}^*}(\mathbf{P}(\mathbf{X}_t)) = \mathbf{P}_t \mathbf{w}^*$

Vector output for each sample

-Learning: $\mathbf{W}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{Y}$

-Prediction: $\mathbf{F}_{\mathbf{W}^*}(\mathbf{P}(\mathbf{X}_t)) = \mathbf{P}_t \mathbf{W}^*$

✓ Ridge Regression

Scalar output for each sample

-Learning: $\mathbf{w}^* = (\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^T \mathbf{y}$

-Prediction: $\mathbf{f}_{\mathbf{w}^*}(\mathbf{P}(\mathbf{X}_t)) = \mathbf{P}_t \mathbf{w}^*$

Vector output for each sample

-Learning: $\mathbf{W}^* = (\mathbf{P}^T \mathbf{P} + \lambda \mathbf{I})^{-1} \mathbf{P}^T \mathbf{Y}$

-Prediction: $\mathbf{F}_{\mathbf{W}^*}(\mathbf{P}(\mathbf{X}_t)) = \mathbf{P}_t \mathbf{W}^*$

✓ 2 Evaluation Metrics for Regression Task

1. Polynomial Regression for Classification:

① Binary Classification

Input: $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$

Output: $f_W(x) = f_W(P(x)) = \text{sign}(P^T W)$

↓
Threshold Function

If $P^T P$ is invertible, then

- Learning: $W^* = (P^T P)^{-1} P^T y$

- Prediction: $f_{W^*}(P(x)) = \text{sign}(P^T W^*) = \begin{bmatrix} \text{sign}((P_1^T)^T W^*) \\ \vdots \\ \text{sign}((P_m^T)^T W^*) \end{bmatrix}$

2. Multi-category Classification

Input: $(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$ y_i is K -dimensional one-hot

Output: $f_W(x) = f_W(P(x)) = \underset{k=1 \dots K}{\text{argmax}} (P^T W)$ encoded vector

Learning: $W^* = (P^T P)^{-1} P^T Y$

Prediction: $f_{W^*}(P(X_t)) = \begin{cases} \arg\max_{k=1 \dots K} ((P_1^t)^T W^*) \\ \vdots \\ \arg\max_{k=1 \dots K} ((P_m^t)^T W^*) \end{cases}$

3. Not recommended for classification:

① Hard threshold function, e.g. 'sign()' "

② $P^T W$ is a number between $-\infty$ and ∞

4. Logistic Regression (With soft threshold function)

Just used for classification! Not Regression!!!

① Binomial Logistic Regression:

Input: $(X_1, y_1) \dots (X_n, y_n)$, $y_i \in [0, 1]$

Output: $f_w(x) = \sigma(x^T w)$, where $\sigma(z) = \frac{1}{1 + e^{-z}}$

Minimize: $J(w) = \frac{1}{N} \sum_{i=1}^N \text{Loss}(f_w(x_i), y_i)$

where $\text{Loss}(f_w(x_i), y_i) = -y_i \ln(f_w(x_i)) - (1 - y_i) \ln(1 - f_w(x_i))$

We use gradient descent! $\nabla_w J(w) = \frac{1}{N} X^T (f_w(X) - y)$

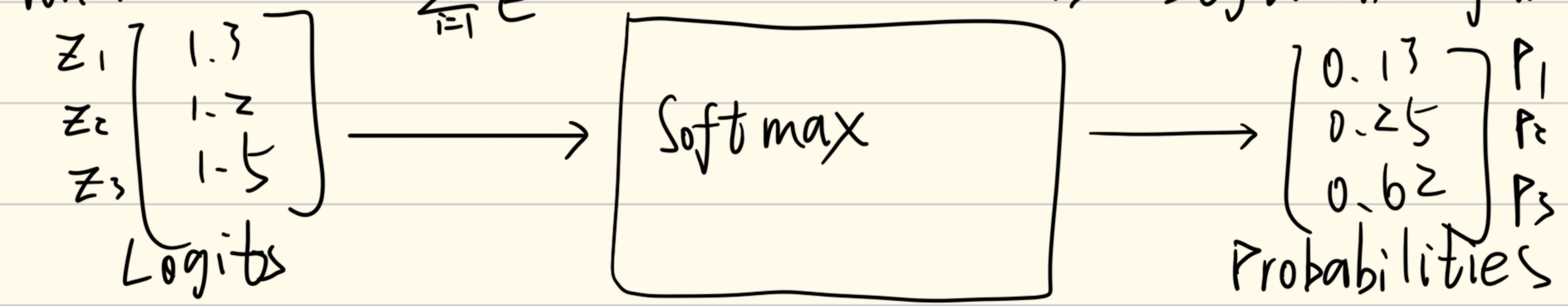
K-class

② Multi-class

Output: $f_w(x) = \sigma(x^T w) = \sigma([x_1 \dots x_d] \begin{bmatrix} w_{0,1} & \dots & w_{0,k} \\ \vdots & & \vdots \\ w_{d,1} & & w_{d,k} \end{bmatrix})$

$= \sigma([x^T w_1 \dots x^T w_k]) = \sigma([z_1 \dots z_k]) = \left[\frac{e^{z_1}}{\sum_{i=1}^k e^{z_i}} \dots \frac{e^{z_k}}{\sum_{i=1}^k e^{z_i}} \right]$

where $\sigma(z)_k = \frac{e^{z_k}}{\sum_{i=1}^k e^{z_i}}$, $k = 1 \dots K$ is softmax function



Evaluation Metrics for Classification

➤ Multicategory Classification

Confusion Matrix for Multicategory Classification

	$P_{\hat{1}}$ (predicted)	$P_{\hat{2}}$ (predicted)		$P_{\hat{C}}$ (predicted)
P_1 (actual)	$P_{1,\hat{1}}$	$P_{1,\hat{2}}$	...	$P_{1,\hat{C}}$
P_2 (actual)	$P_{2,\hat{1}}$	$P_{2,\hat{2}}$	...	$P_{2,\hat{C}}$
⋮	⋮	⋮	⋮	⋮
P_C (actual)	$P_{C,\hat{1}}$	$P_{C,\hat{2}}$		$P_{C,\hat{C}}$

```
from sklearn.metrics import confusion_matrix, accuracy_score  
cm = confusion_matrix(y_true, y_pred)  
acc = accuracy_score(y_true, y_pred)
```

Summary

✓ Polynomial Regression for Classification

Binary Classification

-Learning:

$$\mathbf{w}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{y} \quad y_i \in \{-1, +1\}$$

-Prediction:

$$\mathbf{f}_{\mathbf{w}^*}(\mathbf{P}(\mathbf{X}_t)) = \text{sign}(\mathbf{P}_t \mathbf{w}^*)$$

Multi-category Classification

-Learning:

$$\mathbf{W}^* = (\mathbf{P}^T \mathbf{P})^{-1} \mathbf{P}^T \mathbf{Y}$$

$$\text{-Prediction: } \mathbf{F}_{\mathbf{W}^*}(\mathbf{P}(\mathbf{X}_t)) = \begin{bmatrix} \underset{k=1, \dots, K}{\text{argmax}}((\mathbf{p}_1^t)^T \mathbf{W}^*) \\ \vdots \\ \underset{k=1, \dots, K}{\text{argmax}}((\mathbf{p}_m^t)^T \mathbf{W}^*) \end{bmatrix}$$

✓ Logistic Regression

Binary Classification

-Learning: Gradient Descent

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \frac{1}{N} \mathbf{X}^T (\mathbf{f}_{\mathbf{w}}(\mathbf{X}) - \mathbf{y})$$

-Prediction:

$$\mathbf{f}_{\mathbf{w}^*}(\mathbf{X}_t) = \sigma(\mathbf{X}_t \mathbf{w}^*) = \begin{bmatrix} \sigma((\mathbf{x}_1^t)^T \mathbf{w}^*) \\ \vdots \\ \sigma((\mathbf{x}_m^t)^T \mathbf{w}^*) \end{bmatrix}$$

Multi-category Classification

-Learning: Gradient Descent

$$\nabla_{\mathbf{W}} J(\mathbf{W}) = \frac{1}{N} \mathbf{X}^T (\mathbf{F}_{\mathbf{W}}(\mathbf{X}) - \mathbf{Y})$$

-Prediction:

$$\mathbf{F}_{\mathbf{W}^*}(\mathbf{X}_t) = \boldsymbol{\sigma}(\mathbf{X}_t \mathbf{W}^*) = \begin{bmatrix} \boldsymbol{\sigma}((\mathbf{x}_1^t)^T \mathbf{W}^*) \\ \vdots \\ \boldsymbol{\sigma}((\mathbf{x}_m^t)^T \mathbf{W}^*) \end{bmatrix}$$

✓ Evaluation Metrics for Classification Task

1. Types of clusters:

- ① Well-spread cluster
- ② Center-based cluster
- ③ Continuous cluster

2. Types of clustering:

- ① Hard Clustering: Each data point belongs to 1 cluster completely or not. e.g. K-means
- ② Soft Clustering: Each data point belongs to each cluster with a probability or degree of membership. e.g. Fuzzy C-means

3. K-means clustering

- ① Objective function: $J(w, c) = \sum_{i=1}^N \sum_{k=1}^K W_{ik} \|X_i - C_k\|^2$ Squared Euclidean Distance
- ② Limitations: (i) Need to predifine K (ii) Sensitive to initial centroid

selection (iii) Assume spherical, equal sized, similar-density clusters

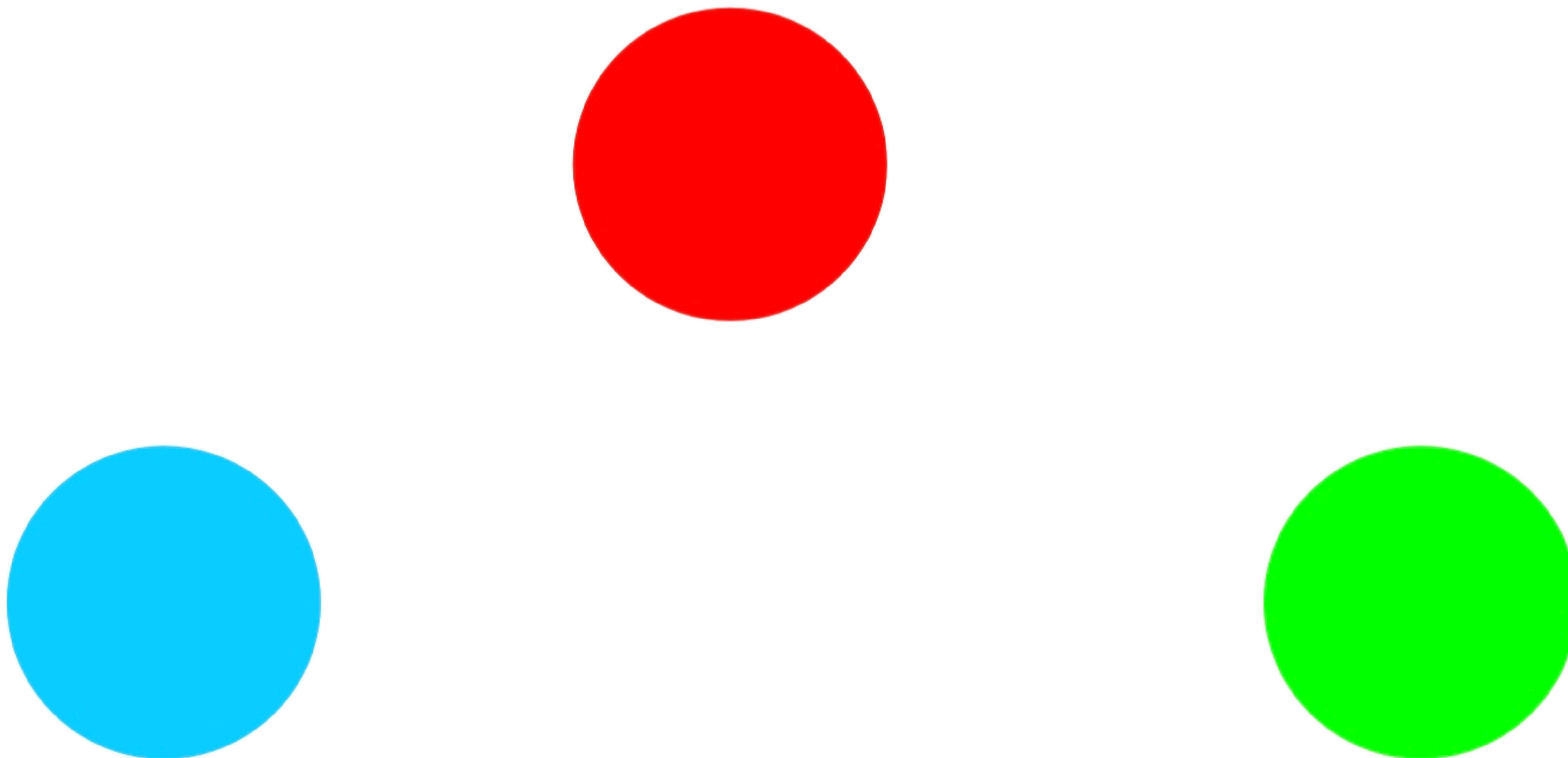
4. Fuzzy C-Means Clustering

① Objective Function: $J(W, c) = \sum_{i=1}^N \sum_{k=1}^C (w_{ik})^r \|x_i - c_k\|^2$

$r > 1$ is the fuzzier, determines the level of cluster fuzziness
(Usually $1.25 \leq r \leq 2$) $\rightarrow$ Overlapping - Uncertainty

Types of Clusters

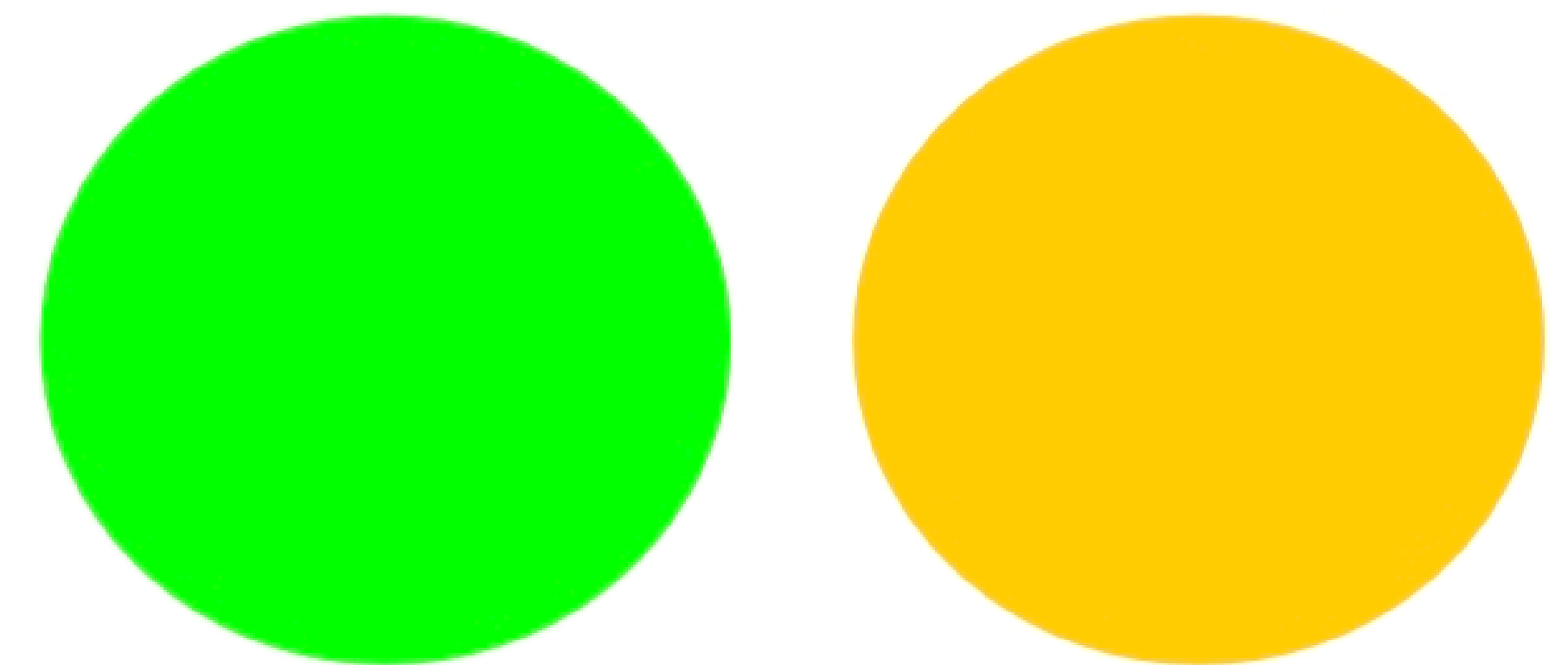
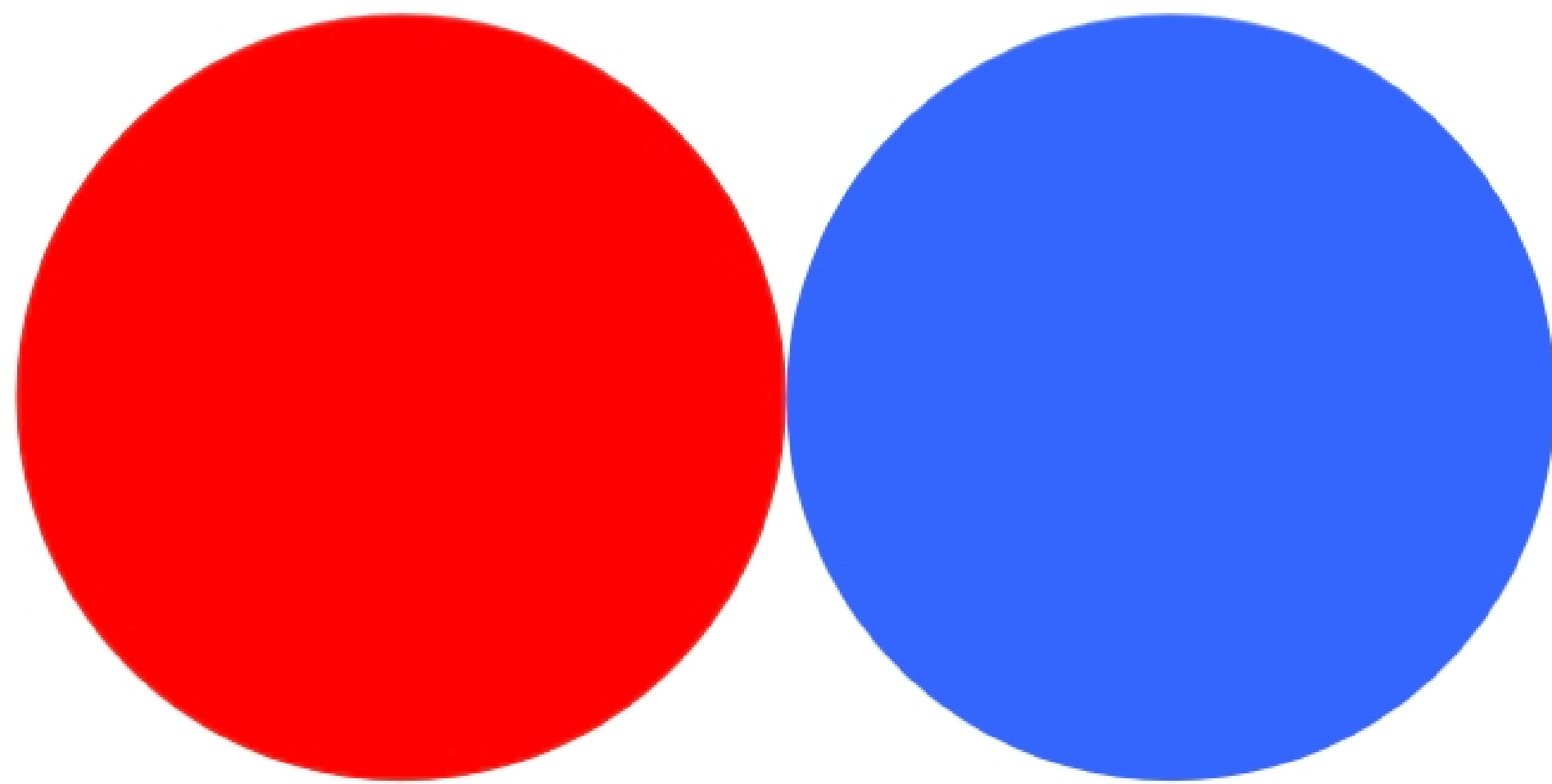
- Well-Separated Cluster:
 - A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster.



3 well-separated clusters

Types of Clusters

- Center-based Cluster:
 - A cluster is a set of objects such that a point in a cluster is closer (more similar) to the “center” of a cluster, than to the center of any other cluster
 - The center of a cluster is often a centroid, the average of all the points in the cluster, or the most “representative” point of a cluster



4 center-based clusters

How many well-separated clusters?

2

K-means Clustering

➤ Algorithm

Looping between **Assignment** and **Centroid Update**

-
- 1: Select K points as the initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning all points to the closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** The centroids don't change
-

K-means Clustering

➤ Formal Formulation

1. Assignment Step (Fix $\mathbf{C}_{K \times d}$ and update $\mathbf{W}_{N \times K}$)

$$\text{If } \|\mathbf{x}_i - \mathbf{c}_k\|^2 < \|\mathbf{x}_i - \mathbf{c}_{\forall j \neq k}\|^2 : w_{ik} = 1 \\ (\text{else } w_{ik} = 0)$$

$$i = 1, \dots, N; j, k = 1, \dots, K$$

2. Update Step (Fix $\mathbf{W}_{N \times K}$ and update $\mathbf{C}_{K \times d}$)

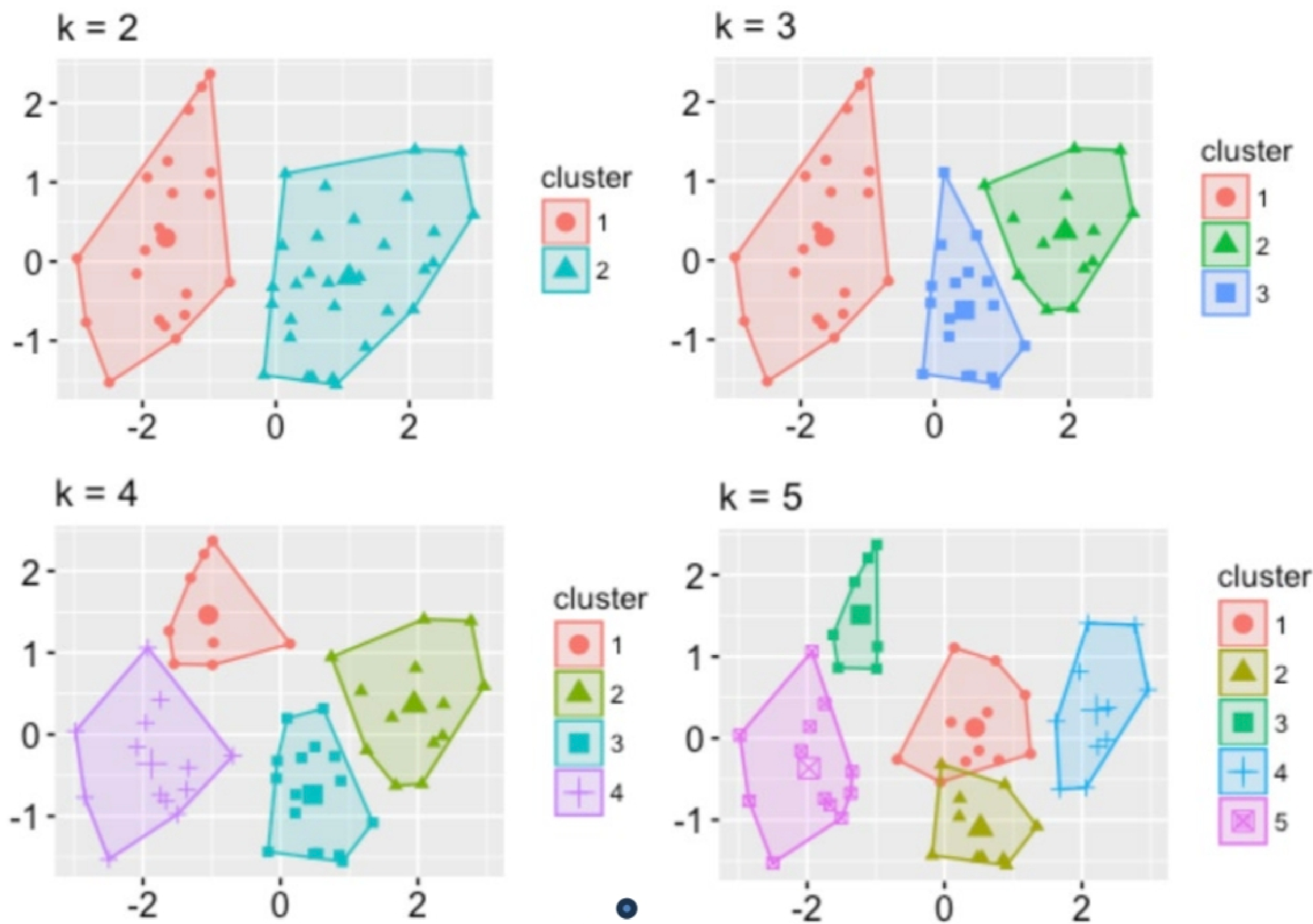
$$\frac{dJ(\mathbf{W}, \mathbf{C})}{d\mathbf{C}} = \begin{bmatrix} \frac{\partial \sum_{i=1}^N \sum_{k=1}^K w_{ik} (\mathbf{x}_i - \mathbf{c}_k)^T (\mathbf{x}_i - \mathbf{c}_k)}{\partial \mathbf{c}_1} \\ \vdots \\ \frac{\partial \sum_{i=1}^N \sum_{k=1}^K w_{ik} (\mathbf{x}_i - \mathbf{c}_k)^T (\mathbf{x}_i - \mathbf{c}_k)}{\partial \mathbf{c}_K} \end{bmatrix} = \begin{bmatrix} -2 \sum_{i=1}^N w_{i1} (\mathbf{x}_i - \mathbf{c}_1) \\ \vdots \\ -2 \sum_{i=1}^N w_{iK} (\mathbf{x}_i - \mathbf{c}_K) \end{bmatrix} = \mathbf{0}$$

$$\mathbf{c}_k = \frac{\sum_{i=1}^N w_{ik} \mathbf{x}_i}{\sum_{i=1}^N w_{ik}}, k = 1, \dots, K$$

中心点是各个点的平均值

K-means Clustering: Limitations

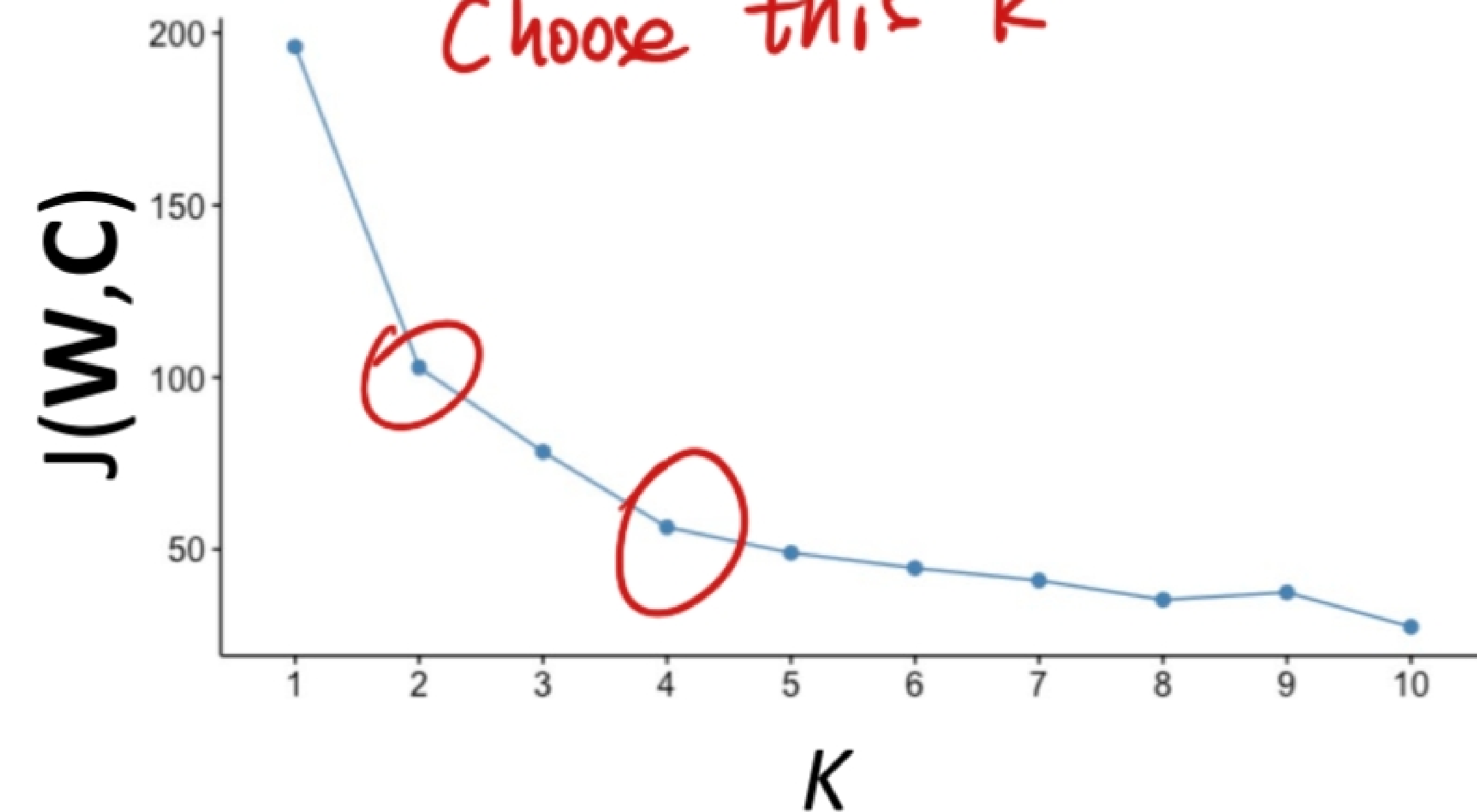
➤ Need to predefine K



Elbow Method:

1. K-means for different K
2. For each K , calculate $J(\mathbf{W}, \mathbf{C})$
3. Plot the curve of $J(\mathbf{W}, \mathbf{C})$ over K
4. "bend" or "knee"

Choose this k



What is the optimal K ?

Fuzzy C-Means Clustering

➤ Algorithm Formulation

1. Assignment Step (Fix $\mathbf{C}_{C \times d}$ and update $\mathbf{W}_{N \times C}$)

$$w_{ik} = \frac{1}{\sum_{j=1}^C \left(\frac{\|\mathbf{x}_i - \mathbf{c}_k\|}{\|\mathbf{x}_i - \mathbf{c}_j\|} \right)^{\frac{2}{r-1}}}$$

更新 update

2. Update Step (Fix $\mathbf{W}_{N \times C}$ and update $\mathbf{C}_{C \times d}$)

$$\frac{dJ(\mathbf{W}, \mathbf{C})}{d\mathbf{C}} = \begin{bmatrix} \frac{\partial \sum_{i=1}^N \sum_{k=1}^C (w_{ik})^r (\mathbf{x}_i - \mathbf{c}_k)^T (\mathbf{x}_i - \mathbf{c}_k)}{\partial \mathbf{c}_1} \\ \vdots \\ \frac{\partial \sum_{i=1}^N \sum_{k=1}^C (w_{ik})^r (\mathbf{x}_i - \mathbf{c}_k)^T (\mathbf{x}_i - \mathbf{c}_k)}{\partial \mathbf{c}_K} \end{bmatrix} = \begin{bmatrix} -2 \sum_{i=1}^N (w_{i1})^r (\mathbf{x}_i - \mathbf{c}_1) \\ \vdots \\ -2 \sum_{i=1}^N (w_{iC})^r (\mathbf{x}_i - \mathbf{c}_K) \end{bmatrix} = \mathbf{0}$$

$$\mathbf{c}_k = \frac{\sum_{i=1}^N (w_{ik})^r \mathbf{x}_i}{\sum_{i=1}^N (w_{ik})^r}, k = 1, \dots, C$$

1. Activation Function: sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$ ReLU: $\sigma(z) = \max(0, z)$
(Non-linear)

2. Weight Matrix: $W^{(l)} = \begin{bmatrix} W_{0,1}^{(l)} & \dots & W_{0,D_l}^{(l)} \\ \vdots & & \vdots \\ W_{D_{l-1},1}^{(l)} & \dots & W_{D_{l-1},D_l}^{(l)} \end{bmatrix} = \begin{bmatrix} W_1^{(l)} & \dots & W_{D_l}^{(l)} \end{bmatrix}$
↓
Weight vector for first neuron in layer l

$$3. o^{(l)} = \sigma^{(l)}([1, o^{(l-1)}] W^{(l)})$$

A MLP is essentially a nested function:

$$f_W(x) = \sigma^{(L)}([1, \dots, \sigma^{(2)}([1, \sigma^{(1)}(x^T W^{(1)})] W^{(2)}) \dots] W^{(L)})$$

4. Why non-linear activation function?

With linear/affine $\sigma(z)$, MLP = single layer perceptron

5. MLP for image processing limitation:

① Data explosion

② Loss of spacial structure

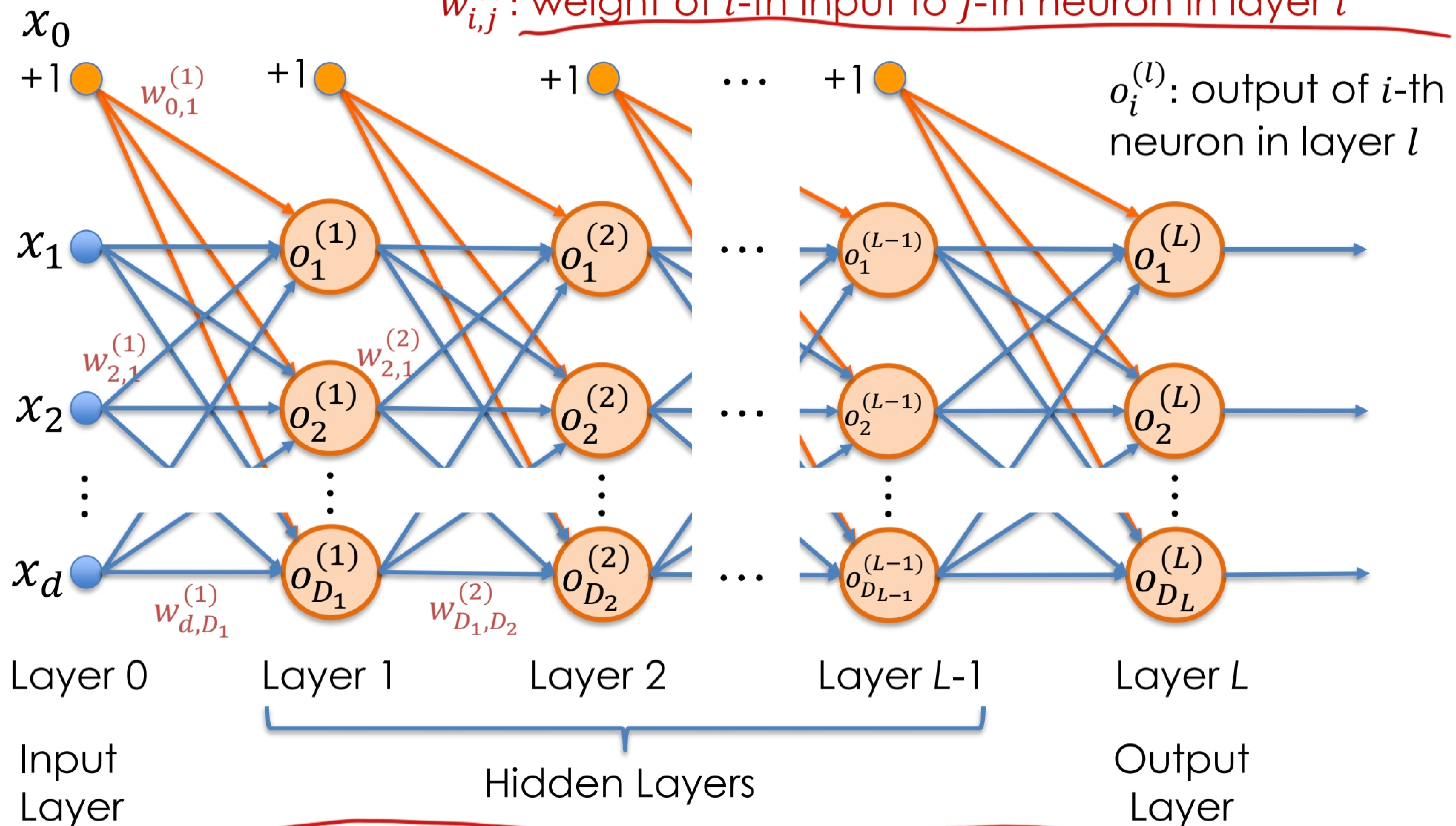
$$b. W^l = \begin{bmatrix} - & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$$

Rows: The number of input

Columns: The number of output

Architecture

$w_{i,j}^{(l)}$: weight of i -th input to j -th neuron in layer l



Note: D_l denotes the number of neurons in layer l

Multilayer Perceptron (MLP)

Learning (W)

1. Random Initialization of **W**

2. Forward Pass (fixed **W**)

i. Compute output responses

3. **Backward Pass** (update **W**)

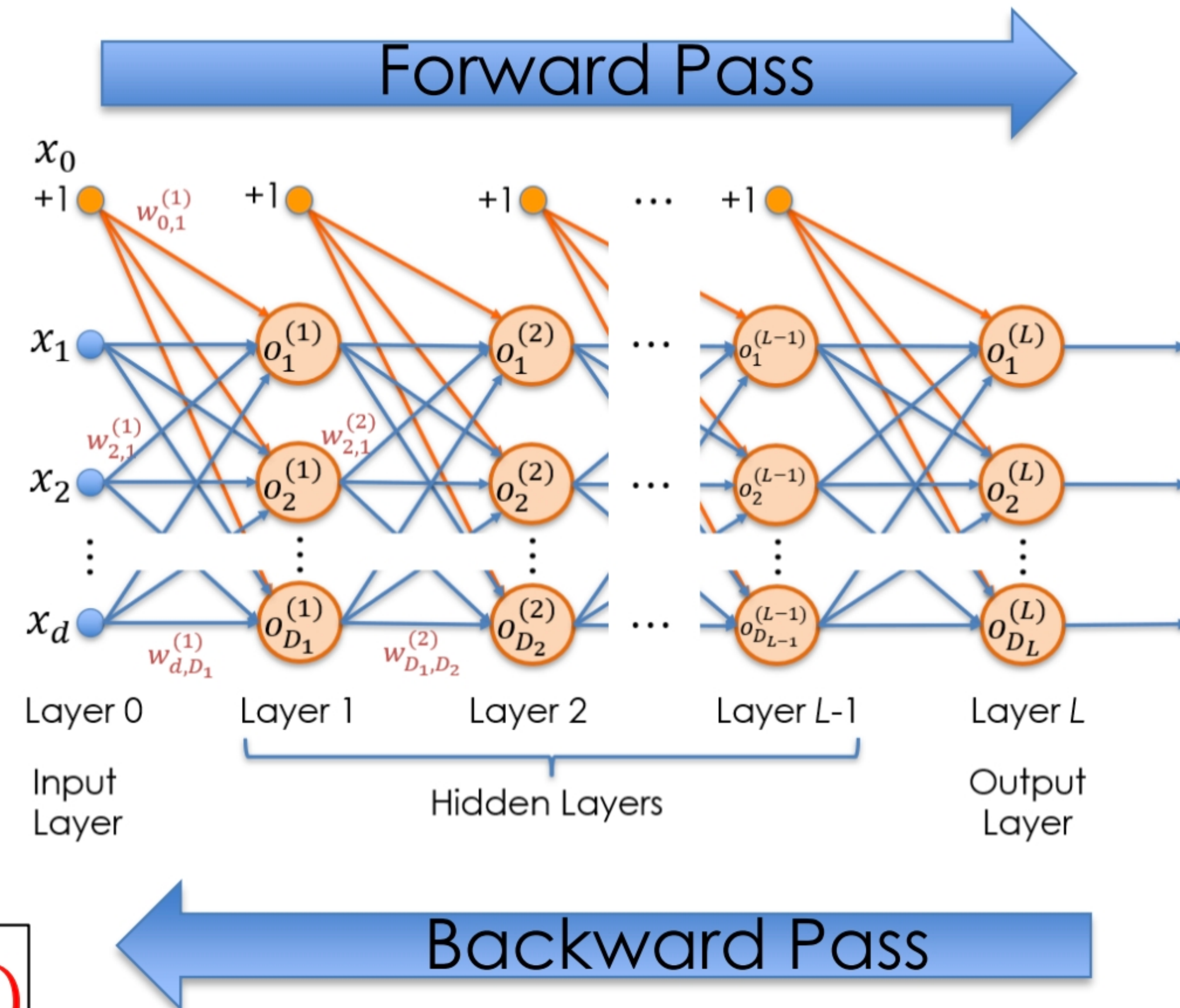
i.e., **Backpropagation**

i. Propagate the errors backward

ii. Update **W**

$$\mathbf{W}_{new} = \mathbf{W}_{old} - \eta \nabla_{\mathbf{W}} J(\mathbf{W}_{old})$$

In essence, Backpropagation = Gradient Decent!



Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

$$\mathbf{W}_{new} = \mathbf{W}_{old} - \eta \nabla_{\mathbf{W}} J(\mathbf{W}_{old})$$

$$\nabla_{\mathbf{W}} J(\mathbf{W}_{old}) = \frac{1}{N} \sum \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{W}_{old})$$

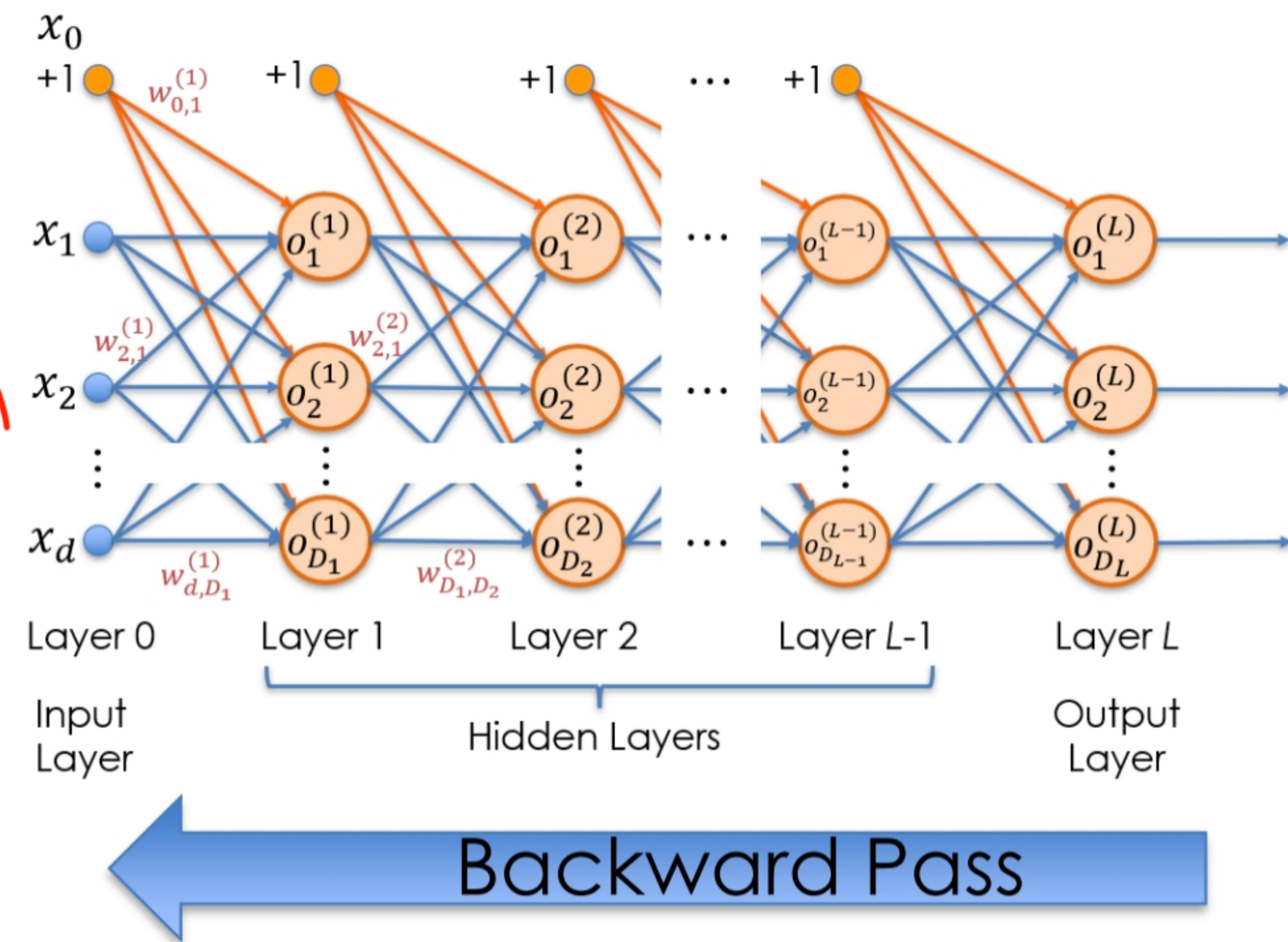
Loss function

One of them

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{i,j}^{(l)}}$$

where $z_j^{(l)} = (\mathbf{a}^{(l)})^T \mathbf{w}_j^{(l)}$

$\mathbf{a}^l$: input vector to layer l = $\begin{cases} \mathbf{x}, & \text{if } l = 1 \\ [1, \mathbf{o}^{(l-1)}]^T, & \text{else} \end{cases}$



Multilayer Perceptron (MLP)

➤ Learning (W): Backpropagation

If output layer ($l=L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \sum_{m=1}^{D_L} \mathcal{L}'(o_m^{(L)}) \sigma^{(L)'}(z_j^{(L)}) a_i^{(L)}$$

If softmax activation, categorical cross entropy loss:

$$\frac{d\mathcal{L}}{d\mathbf{W}^{(L)}} = \mathbf{a}^{(L)} \underbrace{(\mathbf{f}_{\mathbf{W}}(\mathbf{x}) - \mathbf{y})}_{\boldsymbol{\epsilon}^{(L)}}, \quad \frac{dJ}{d\mathbf{W}^{(L)}} = \frac{1}{N} (\mathbf{A}^{(L)})^T \underbrace{(\mathbf{F}_{\mathbf{W}}(\mathbf{X}) - \mathbf{Y})}_{\mathbf{E}^{(L)}}$$

If hidden layer ($l < L$):

$$\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \sigma^{(l)'}(z_j^{(l)}) \sum_{m=1}^{D_{l+1}} \epsilon_m^{(l+1)} w_{j,m}^{(l+1)} a_i^{(l)}$$

If ReLU activation:

$$\frac{d\mathcal{L}}{d\mathbf{W}^{(l)}} = \mathbf{a}^{(l)} (\boldsymbol{\delta}_{\mathbf{Z}^{(l)}} * (\boldsymbol{\epsilon}^{(l+1)} (\mathbf{W}_{1:D_l,:}^{(l+1)})^T))$$

$$\frac{dJ}{d\mathbf{W}^{(l)}} = \frac{1}{N} (\mathbf{A}^{(l)})^T (\boldsymbol{\delta}_{\mathbf{Z}^{(l)}} * (\mathbf{E}^{(l+1)} (\mathbf{W}_{1:D_l,:}^{(l+1)})^T))$$

MLP for Image Classification

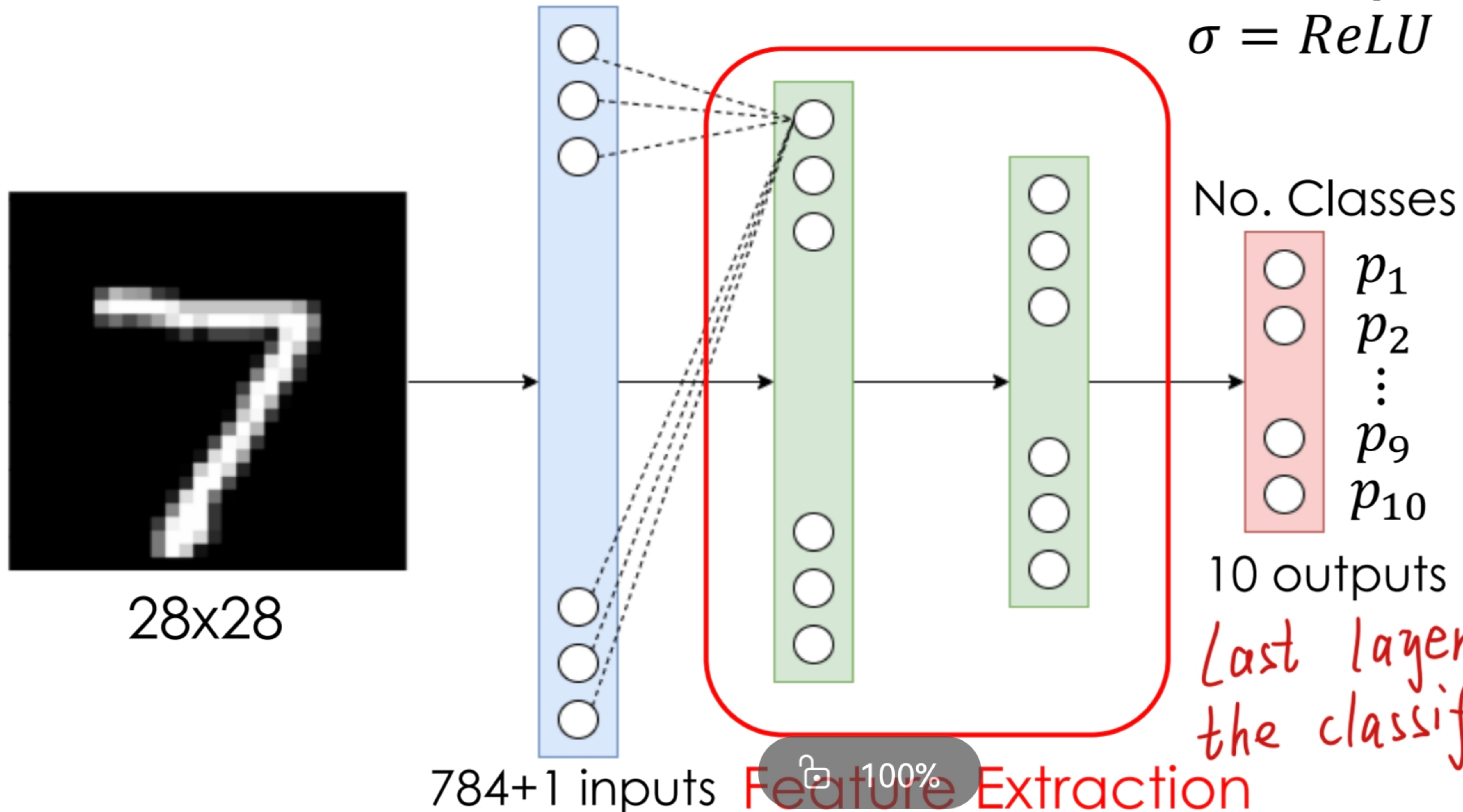
If output layer ($l=L$):

$$\sigma = \text{Softmax}$$

If hidden layer ($l < L$):

$$\sigma = \text{ReLU}$$

No. input features



Last layer does the classification.